클라우드 엔지니어링 6기

멋쟁이사자처럼

26.03.30 (월) | 정범석님

자바 변수, 형변환, 연산자 정리

👉 오늘의 강의 학습 요약

본 강의는 자바 개발 환경(JDK/JRE, 컴파일, 실행 흐름, JVM, 바이트코드)과 자바 코드의 기본 구성 요소(식별자, 키워드, `main` 메소드 시그니처, API/문서)를 빠르게 복습한 뒤, 데이터 타입(기본형/참조형)과 출력 함수(`print`, `println`, `printf`)를 기반으로 변수 개념을 체계적으로 확장합니다.

이어서 변수의 선언, 초기화, 사용, 변경을 실습으로 확인하고, 메모리 관점에서 기본형/참조형 저장 방식 차이, 변수 선언 위치에 따른 로컬/인스턴스/클래스(스태틱) 변수의 생성, 소멸, 저장 메모리(Stack/Heap) 및 초기화 규칙을 정리합니다. 마지막으로 상수(`final`), 데이터 형 변환(묵시적/명시적), 그리고 연산자(산술/대입/비교/논리/증감)를 예제로 검증하며, 숏서킷 논리 연산과 전치, 후치 증감의 차이를 강조합니다.

1. 자바 실행 구조와 기본 문법

> **JDK 설치와 환경변수 설정을 기반으로 `javac` 로 컴파일해 바이트코드를 만들고, `java` 로 JVM에서 실행하며, 자바 클래스, 식별자, 키워드, `main` 시그니처와 API 문서 활용을 기본기로 정리합니다.**

자바 개발을 위해서는 JDK 설치와 환경변수 설정이 전제됩니다. 자바는 소스 코드를 바로 실행하는 형태가 아니라, 컴파일을 통해 클래스 파일(바이트코드)을 만들고 JVM에서 실행하는 구조입니다.

컴파일과 실행은 IDE가 대부분 대행하지만, GUI가 없는 환경에서는 명령어 기반으로 처리해야 하므로 기본 도구의 역할을 이해해야 합니다. 컴파일러는 `javac` , 실행기는 `java` 로 구분됩니다.

자바는 바이트코드가 JVM 위에서 동작하므로 운영체제에 덜 의존적으로 실행됩니다. 즉 "한 번 컴파일하면 여러 환경에서 실행"이 가능하다는 점이 핵심 특징으로 정리됩니다.

JDK 내부는 하드웨어/OS 상단에 JVM이 위치하고, 그 위에서 바이트코드와 각종 유틸리티가 동작하는 형태로 이해할 수 있습니다. 실행만 필요하면 JRE, 개발까지 포함하면 JDK가 필요하다는 구분이 중요합니다.

자바 클래스의 이름과 구성 요소는 식별자 규칙으로 구분합니다. 키워드(예약어)는 자바에서 특별한 의미로 미리 정의된 단어이며, 그 외는 사용자 정의 식별자로 이름을 부여할 수 있습니다.

클래스, 메소드, 변수는 모두 이름을 붙이는 대상이며, 협업을 위한 관례(명명 규칙)를 지키는 것이 중요합니다. 클래스는 첫 글자 대문자, 변수는 소문자 중심 등 관례를 어기면 문법 오류는 아니더라도 가독성과 협업 품질이 떨어집니다.

`main` 메소드는 프로그램의 시작점이며, 시그니처가 정해져 있어야 실행됩니다. 중괄호를 제외한 형태(반환 타입, 접근제어자, 매개변수 등)를 시그니처라고 부릅니다.

자바 코드에서 첫 글자가 대문자인 식별자는 보통 클래스(예: `String` , `System`)로 해석됩니다. 이렇게 미리 만들어진 클래스를 활용해 애플리케이션을 구성하며, 이러한 라이브러리 집합을 API로 부릅니다.

API는 문서(Java API Documentation) 형태로 제공되며, 개발자는 이를 참고하여 이미 구현된 기능을 재사용합니다. API 문서를 자주 확인하는 습관이 생산성과 스킬업에 직접 연결됩니다.

핵심 키워드 `#JDK` `#JRE` `#JVM` `#바이트코드` `#main시그니처`

2. 데이터 타입과 출력(리터럴, 이스케이프 포함)

> **자바 데이터는 기본형 8가지와 참조형으로 나뉘며, 리터럴을 `print/println/printf` 로 출력하고 이스케이프 문자를 통해 줄바꿈 등 표현을 제어합니다.**

자바에서 다루는 데이터는 리터럴로 표현되며, 출력은 `print` , `println` , `printf` 세 방식이 핵심입니다. `println` 은 줄바꿈을 포함하고, `print` 는 줄바꿈을 포함하지 않으며, `printf` 는 형식 지정자를 통해 포맷을 구성합니다.

기본형은 총 8가지이며 정수 4개, 실수 2개, 문자 1개, 논리 1개로 구성됩니다. 정수 리터럴은 10진수뿐 아니라 16진수 등 다양한 표기 가능성을 염두에 둬야 하며, `long` 리터럴에는 `L` 접미를 붙여 구분합니다.

실수 리터럴의 기본 타입은 `double` 이므로, `float` 로 저장하려면 `F` 접미가 필요합니다. 이를 생략하면 8바이트 실수(`double`)로 해석되어 `float` 변수에 대입 시 타입 불일치가 발생합니다.

문자(`char`)는 2바이트이며 전 세계 문자를 표현할 수 있습니다. 문자 리터럴은 작은따옴표를 사용하고, 줄바꿈(`\n`) 등 이스케이프 문자는 문자열/출력 제어에서 매우 자주 사용됩니다.

논리형(`boolean`)은 `true` 와 `false` 만 허용됩니다. 다른 언어에서 허용되는 “truthy/falsy” 개념이 자바에는 적용되지 않는다는 점이 비교 포인트입니다.

참조형은 기본형 8개를 제외한 모든 타입이며, 대표적으로 클래스/인터페이스/배열이 포함됩니다. 클래스 기반 데이터도 “데이터로서 처리”된다는 관점이 중요합니다.

핵심 키워드 #기본형 #참조형 #리터럴 #printf #이스케이프문자

3. 변수 기초: 선언, 초기화, 사용, 변경과 메모리

> **변수는 데이터 저장과 재사용을 위해 선언→초기화→사용으로 다루며, 기본형은 값 자체를 저장하고 참조형은 주소값을 저장해 실제 데이터는 다른 영역에 존재합니다.**

변수는 데이터를 저장하는 공간이며, 저장 목적의 핵심은 재사용입니다. 동일한 리터럴을 반복 작성하는 대신 변수에 저장해 여러 번 활용하는 형태가 일반적입니다.

변수 생성은 3단계로 정리됩니다. 첫째는 선언(영역 확보), 둘째는 초기화(처음 값 저장), 셋째는 사용(출력/연산 등)입니다.

변수 선언 문법은 `데이터형 변수명` 입니다. 예를 들어 `int num;` 은 4바이트 크기의 정수를 담을 영역을 확보하고, 해당 영역을 `num` 이라는 이름으로 참조할 수 있게 합니다.

초기화 문법은 `변수명 = 값` 이며, 비어 있는 변수 공간에 최초 값을 저장하는 작업입니다. 이후에는 변수를 통해 값을 출력하거나 연산에 활용할 수 있습니다.

변수는 "변할 수 있다"는 특성 때문에, 동일 변수에 다른 값을 재할당하여 변경할 수 있습니다. 같은 변수라도 시점에 따라 다른 값이 들어갈 수 있음을 코드 흐름으로 이해해야 합니다.

기본형 변수는 변수 공간에 값 자체가 저장됩니다. 반면 참조형 변수는 값(예: 문자열) 자체가 아니라 "주소값"을 저장하며, 실제 데이터는 다른 메모리 영역에 저장됩니다.

참조형 변수 크기는 데이터 내용과 무관하게 "주소를 담는 크기"로 고정이라는 점을 강조합니다. 강의에서는 참조형 변수 크기를 4바이트로 정리했습니다.

핵심 키워드 #변수선언 #초기화 #기본형변수 #참조형변수 #주소값

4. 변수 실습과 명명 규칙, 출력 포맷

> **실무 관례에 맞게 변수는 소문자 중심의 의미 있는 명사로 명명하며, `println` 연결(`+`)과 `printf` 서식 지정자로 여러 타입을 안전하게 출력합니다.**

여러 데이터(이름, 나이, 주소, 키, 몸무게, 결혼 여부)를 저장하려면 각 데이터에 맞는 타입을 선택해 변수를 선언해야 합니다. 문자열은 `String` , 정수는 `int` , 실수는 `float/double` , 논리는 `boolean` 으로 분류합니다.

변수 명명은 관례적으로 소문자 중심이며, 의미 있는 명사형으로 작성합니다. 변수명을 대문자로 시작하면 클래스 명명 규칙과 충돌해 혼동을 유발할 수 있습니다.

실수 리터럴을 `float` 에 대입할 때 `F` 접미가 없으면 기본 `double` 로 해석되어 오류가 발생합니다. 해결은 타입을 `double` 로 올리거나, 리터럴을 `float` 로 명시(`183.3F`)하는 방식입니다.

출력은 `println` 에서 문자열 결합을 위해 `+` 를 사용합니다. 이때 `+` 는 수치 덧셈이 아니라 "문자열 연결"로 동작할 수 있음을 이해해야 합니다.

`printf` 는 형식 지정자를 사용해 값 치환을 수행합니다. 줄바꿈이 필요하면 `\n` 을 명시해 출력 라인을 제어합니다.

출력 방식	특징	예시 포인트
`println`	자동 줄바꿈이 포함됩니다.	단순 출력에 적합합니다.
`print`	줄바꿈이 포함되지 않습니다.	연속 출력에 적합합니다.
`printf`	포맷 문자열과 형식 지정자를 사용합니다.	`%s` , `%d` 와 `\n` 조합이 핵심입니다.

핵심 키워드 #명명규칙 #String #float접미사F #문자열연결 #printf서식

5. 변수 종류: 로컬, 인스턴스, 클래스(스태틱)

> **변수는 선언 위치(메소드 안/밖)와 `static` 유무로 로컬, 인스턴스, 클래스 변수로 구분되며, 생성, 소멸 시점과 저장 메모리(Stack/Heap), 초기화 규칙이 달라집니다.**

변수는 선언 위치에 따라 분류됩니다. 기준은 메소드이며, 메소드 블록 내부에 선언되면 로컬 변수, 메소드 밖(클래스 블록)에 선언되면 인스턴스 변수 또는 클래스 변수 후보가 됩니다.

메소드의 매개변수(괄호 안에 선언되는 변수)도 메소드 내부 범위에 포함되므로 로컬 변수 범주로 정리됩니다. 즉 로컬 변수는 메소드 블록 내부와 파라미터를 포함하는 개념입니다.

메소드 밖 변수 중 `static` 이 붙으면 클래스(스태틱) 변수, 붙지 않으면 인스턴스 변수로 구분됩니다. 코드를 봤을 때 이 구분이 가능해야 이후 객체/클래스 학습이 안정됩니다.

로컬 변수는 메소드가 호출(실행)될 때 생성되고, 메소드 블록이 끝나면 소멸됩니다. 따라서 호출 횟수만큼 생성, 소멸이 반복되는 "짧은 생명주기"의 변수입니다.

로컬 변수는 스택(Stack) 메모리에 저장됩니다. 스택은 입구가 하나인 구조로, 메소드 실행 흐름에 따라 변수가 쌓였다가 블록 종료 시 제거되는 형태로 이해합니다.

로컬 변수는 사용 전에 반드시 초기화해야 합니다. 초기화되지 않은 로컬 변수를 사용하면 컴파일 단계에서 오류가 발생하며, 이는 안전한 실행을 위한 언어 차원의 제약입니다.

인스턴스 변수는 객체 생성(`new`) 시점에 생성됩니다. 객체가 힙(Heap) 메모리에 생성되면서 인스턴스 변수와 메소드가 포함된 인스턴스 상태가 구성됩니다.

인스턴스 변수는 자동 초기화가 적용됩니다. 정수는 0, 실수는 0.0, 논리값은 `false` , 문자는 `\u0000` (널 문자), 참조형은 `null` 로 초기화된다고 정리합니다.

인스턴스의 소멸은 개발자가 직접 `delete` 로 처리하는 방식이 아니라, 시스템이 "어느 변수도 참조하지 못하는 상태(가비지)"를 기준으로 제거합니다. 참조 변수가 다른 인스턴스를 가리키도록 재할당하거나 `null` 을 대입하면 기존 인스턴스는 떠 있는 상태가 되어 제거 대상이 됩니다.

구분	선언 위치/조건	생성 시점	소멸 시점	저장 메모리	초기화 규칙
로컬 변수	메소드(블록) 안, 파라미터 포함입니다.	메소드 실행 시점입니다.	메소드 블록 종료 시점입니다.	Stack입니다.	반드시 명시적 초기화가 필요합니다.
인스턴스 변수	메소드 밖이며 `static` 이 없습니다.	`new` 로 객체 생성 시점입니다.	참조 불가 상태가 되면 시스템이 제거합니다.	Heap입니다.	자동 초기화가 됩니다.
클래스 변수	메소드 밖이며 `static` 이 있습니다.	본 강의에서는 상세를 이후로 미룹니다.	본 강의에서는 상세를 이후로 미룹니다.	본 강의에서는 상세를 이후로 미룹니다.	본 강의에서는 상세를 이후로 미룹니다.

핵심 키워드 #로컬변수 #인스턴스변수 #static변수 #스택메모리 #힙메모리

6. 변수 공통 특징: 중복 금지와 블록 스코프

> **변수명은 같은 블록 내에서 중복될 수 없고, 자바는 중괄호 블록 단위로 식별 범위가 결정되는 블록 스코프 언어입니다.**

변수명은 동일 블록(스코프) 내에서 중복될 수 없습니다. 타입이 달라도 변수명만으로 식별해야 하므로 중복 선언은 식별 불가능 문제를 유발합니다.

자바는 블록 스코프를 사용합니다. 중괄호 `{}` 내부에서 선언된 변수는 해당 블록 내부에서만 인식되며, 블록 밖에서 접근하면 범위 오류가 발생합니다.

같은 블록에서는 중복이 금지되지만, 블록이 다르면 동일 이름을 사용할 수 있습니다. 또한 인스턴스 변수와 로컬 변수는 서로 다른 영역/스코프에 존재하므로 이름이 같아도 구분되는 경우가 있습니다.

핵심 키워드 #변수명중복 #블록스코프 #스코프 #중괄호블록 #식별자

7. 상수: final과 활용 의의

> **`final` 을 사용하면 값이 변경 불가능한 상수가 되며, 변수와 달리 재할당이 컴파일 단계에서 차단되어 안정성과 정확성을 높입니다.**

상수는 값이 변하지 않는다는 점에서 변수와 구분됩니다. 자바에서는 `final` 키워드를 사용해 상수를 선언하며, 관례적으로 이름을 대문자로 작성해 변수와 시각적으로 구별합니다.

상수 문법은 `final 데이터타입 상수명 = 값;` 형태로 정리됩니다. 상수에 재할당을 시도하면 즉시 오류가 발생하므로 "변경되면 안 되는 값"을 강제할 수 있습니다.

상수는 요일, 학년, 역할(Role)처럼 허용 값이 정해져 있고 변경되면 정확성이 깨지는 영역에서 특히 유용합니다. 단순히 "변경하지 않겠다"는 약속을 변수로 표현하는 것보다 언어 기능으로 강제하는 것이 리스크를 줄입니다.

상수 개념은 확장되어 더 구조화된 형태(예: 열거형)로 발전할 수 있으며, 이는 이후 단원에서 다룰 수 있는 연결 지점입니다.

핵심 키워드 #final #상수 #재할당금지 #대문자관례 #고정값

8. 데이터 형 변환: 묵시적, 명시적

> **작은 타입이 큰 타입으로 이동할 때는 묵시적 변환이 일어나며, 큰 타입을 작은 타입에 저장하려면 캐스팅 연산자 `(타입)` 로 명시적 변환이 필요합니다.**

데이터 형 변환은 실행 중 원래 타입을 다른 타입으로 바꾸는 메커니즘입니다. 크게 자동(묵시적) 형 변환과 강제(명시적) 형 변환으로 구분합니다.

묵시적 형 변환은 작은 타입에서 큰 타입으로 저장할 때 자연스럽게 발생합니다. 예를 들어 `byte → short → int → long → float → double` 방향은 대입 시 자동 변환이 가능합니다.

문자(`char`)는 내부적으로 코드값(정수)으로 표현되므로 `int` 로 자동 변환될 수 있습니다. 대문자 `'A'` 는 65, 소문자 `'a'` 는 97이며, 문자 연산이 가능한 이유도 자동 변환으로 숫자 연산이 수행되기 때문입니다.

정수형 중 `int` 보다 작은 타입(`byte` , `short`)은 산술 연산 결과가 `int` 로 승격될 수 있습니다. 따라서 `short + short` 결과를 `short` 에 바로 넣으면 타입 불일치가 발생하고, `int` 로 받거나 캐스팅이 필요합니다.

문자열과 비문자열이 `+` 로 결합되면 결과는 문자열이 됩니다. 이때 `+` 는 산술 덧셈이 아니라 연결 연산자로 동작하며, 연산 순서에 따라 결과 문자열이 달라질 수 있습니다.

명시적 형 변환은 큰 타입을 작은 타입으로 저장할 때 필요합니다. 캐스팅 문법은 `(대상타입)값` 이며, 연산 결과를 캐스팅하려면 우선순위를 위해 괄호로 연산을 묶어야 합니다.

```
short s1 = 10;
short s2 = 20;
// short r = s1 + s2; // 연산 결과는 int로 승격되어 오류가 날 수 있습니다.
short r = (short)(s1 + s2);
```

핵심 키워드 #묵시적형변환 #명시적형변환 #캐스팅 #int승격 #문자코드

9. 연산자: 산술, 대입, 비교, 논리, 증감

> **산술/대입/비교/논리/증감 연산자를 문법과 예제로 정리하며, 정수 나눗셈 특성, 비교, 논리의 boolean 결과, 숏서킷, 전치, 후치 우선순위를 핵심으로 다룹니다.**

산술 연산자는 `+`, `-`, `*`, `/`, `%` 로 구성됩니다. 정수끼리의 나눗셈은 결과가 정수로 떨어지며, 소수 결과를 원하면 한쪽을 실수로 만들어 연산해야 합니다.

대입 연산자 `=` 는 오른쪽 값을 왼쪽 변수에 저장합니다. 축약 대입 연산자(`+=`, `-=`, `*=`, `/=`, `%=`)는 "연산 후 재할당"을 간단히 표현하며, `a = a + b` 를 `a += b` 로 줄이는 형태입니다.

비교 연산자는 값 비교를 수행하며 결과는 항상 `boolean` 입니다. `==`, `!=`, `>`, `<`, `>=`, `<=` 가 기본이며, 비교 결과를 바로 출력하거나 변수에 저장해 재사용할 수 있습니다.

논리 연산자는 `&&` (AND), `||` (OR), `!` (NOT)로 구성되며 결과는 `boolean` 입니다. `&&` 는 모두 참일 때만 참, `||` 는 하나라도 참이면 참, `!` 는 반전입니다.

`&&` 와 `||` 는 숏서킷(Short-circuit) 평가를 수행합니다. 앞의 값만으로 결과가 확정되면 뒤의 식은 실행되지 않으며, IDE에서 데드코드처럼 표시될 수 있습니다.

논리 연산자는 보통 비교 연산식과 함께 사용됩니다. 즉 `true && false` 같은 리터럴 조합보다 `(n > 15) && (n < 30)` 처럼 조건식을 결합하는 형태가 실사용 패턴입니다.

증감 연산자 `++`, `--` 는 값을 1씩 증가/감소시킵니다. 단독 사용 시 전치/후치 결과가 같아 보이지만, 다른 연산(대입, 반환 등)과 결합되면 평가 순서가 달라집니다.

전치(`++n`)는 "먼저 증가 후 사용", 후치(`n++`)는 "먼저 사용 후 증가"로 동작합니다. 예를 들어 `result = ++n` 과 `result = n++` 는 `result` 값이 달라질 수 있습니다.

연산자 분류	대표 연산자	결과 타입/특징
산술	`+ - * / %`	정수 나눗셈은 정수 결과가 기본입니다.
대입	`= += -= *= /= %=`	좌변 변수 값이 누적 변경될 수 있습니다.
비교	`== != > < >= <=`	결과는 항상 `boolean` 입니다.
논리	`&&		
증감	`++ --`	전치/후치는 다른 연산과 결합 시 결과가 달라집니다.

핵심 키워드 #산술연산자 #대입연산자 #비교연산자 #숏서킷 #전치후치